P.Vishwa Nandan(192473015)

TASK A2 – DON'T READ WHAT YOU DON'T NEED (5 MARKS)

1. Aim

To analyse the performance of reading a large attendance log one byte at a time and redesign the program using buffering techniques to improve file-reading efficiency and reduce unnecessary operating-system calls.

2. Problem Description

A program reads a 50 MB attendance log using FileInputStream.read() one byte at a time to count the number of lines.

This approach is inefficient because it performs a separate read operation for every byte.

The program must be redesigned using a buffered stream or byte array buffer to improve performance and reduce the number of read operations.

3. Task A2.1 – Predict the Cost (2 Marks)

3.1 Number of Read Calls

The attendance log file has a size of 50 MB.

When the program uses:

FileInputStream.read()

it reads only one byte per method call.

Assuming 50 MB means 50,000,000 bytes:

Number of read calls:

Number of calls = File Size / Bytes Read Per Call

= 50,000,000 / 1

= 50,000,000 calls

An additional call is normally required to detect the end of the file.

Therefore, the program makes approximately **50,000,001 read calls**.

If the file size is interpreted as 50 MiB, the number of data-byte calls is 52,428,800.

3.2 Why Is It Slow?

Reading one byte at a time is inefficient for the following reasons:

1. Large Number of Read Operations

The program makes approximately 50 million read calls for a 50 MB file.

2. Operating-System Overhead

Each unbuffered FileInputStream read requires interaction with the underlying file-reading implementation, typically resulting in a native read operation.

This creates repeated overhead.

3. Repeated Method Calls

The Java program repeatedly calls read() for every byte instead of processing multiple bytes together.

4. CPU Overhead

The processor spends additional time handling repeated read operations and method calls.

5. Poor Performance for Large Files

As file size increases, the number of read calls also increases proportionally.

3.3 Cost Prediction Table

File Size	Approximate Read Calls
1 MB	1,000,000
10 MB	10,000,000
50 MB	50,000,000

The calculation assumes one byte per call and decimal MB (1 MB = 1,000,000 bytes).

Conclusion: Reading one byte at a time creates unnecessary overhead. Buffering is required to improve file-reading performance.

4. Task A2.2 – Redesign Using Buffering (2 Marks)

4.1 Proposed Solution

Instead of reading one byte at a time, the program should read multiple bytes into a byte array buffer.

For example, a buffer of 64 KB can read 65,536 bytes in one operation.

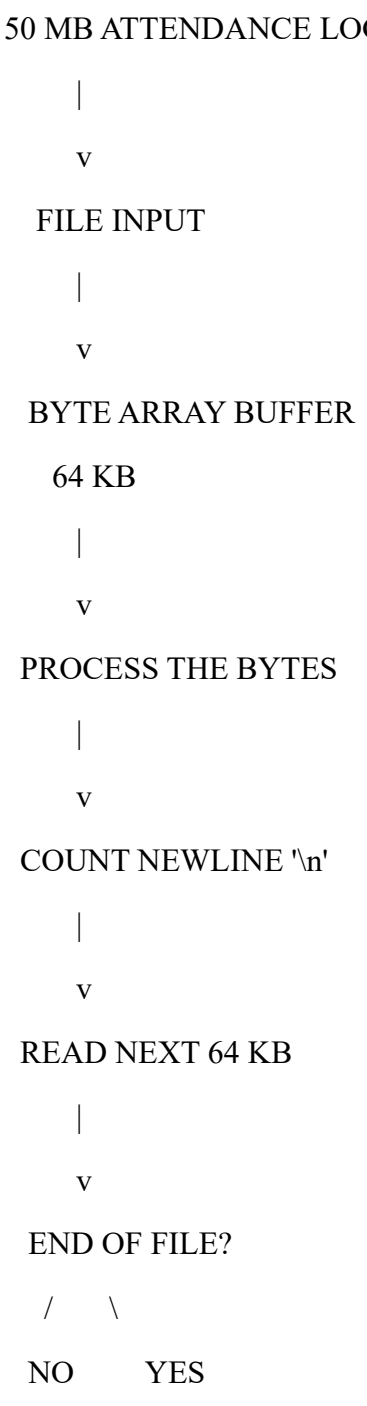
The program processes all the bytes stored in the buffer before requesting the next block.

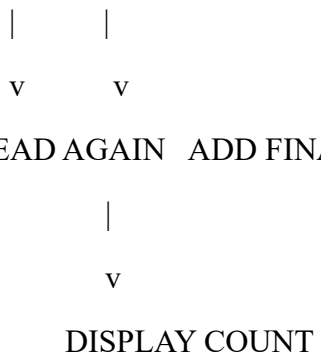
Working

1. Open the attendance log using FileInputStream.
2. Create a byte array buffer of 64 KB.

3. Read multiple bytes into the buffer.
4. Count the newline characters.
5. Repeat until the end of the file.
6. Handle the final line if it does not end with a newline.
7. Close the file.
8. Display the total number of lines.

4.2 Buffered Reading Diagram





4.3 Java Program – BufferedLineCounter.java

This program reads the file in blocks of 64 KB instead of reading one byte at a time.

```
import java.io.FileInputStream;
import java.io.IOException;

public class BufferedLineCounter {

    public static long countLines(String fileName)
            throws IOException {

        // Create a 64 KB buffer
        byte[] buffer = new byte[64 * 1024];

        long lineCount = 0;

        boolean hasData = false;

        int lastByte = -1;

        try (FileInputStream input =
                new FileInputStream(fileName)) {

            int bytesRead;
```

```
        // Read multiple bytes at a time
        while ((bytesRead = input.read(buffer)) != -1) {

            for (int i = 0; i < bytesRead; i++) {

                hasData = true;

                int currentByte = buffer[i] & 0xFF;

                // Count newline characters
                if (currentByte == '\n') {

                    lineCount++;
                }

                lastByte = currentByte;
            }
        }
    }

    // Count the final line if it has no newline
    if (hasData && lastByte != '\n') {

        lineCount++;
    }

    return lineCount;
}
```

```
    public static void main(String[] args) {

        String fileName = "attendance.log";

        try {

            long totalLines = countLines(fileName);

            System.out.println(
                "Total number of lines: " + totalLines
            );

        } catch (IOException e) {

            System.out.println(
                "File reading error: " + e.getMessage()
            );
        }
    }
}
```

4.4 Program Explanation

Step 1 – Creating the Buffer

byte[] buffer = new byte[64 * 1024];

This creates a buffer of 65,536 bytes or 64 KB.

Step 2 – Opening the File

FileInputStream input =

    new FileInputStream(fileName);

This opens the attendance log for reading.

Step 3 – Reading Multiple Bytes

while ((bytesRead = input.read(buffer)) != -1)

This reads multiple bytes into the buffer in one operation.

It continues until the end of the file.

Step 4 – Counting Newlines

```
if (currentByte == '\n') {
   lineCount++;
}
```

Every newline character increases the line count.

Step 5 – Handling the Final Line

```
if (hasData && lastByte != '\n') {
   lineCount++;
}
```

A file may contain a final line without a newline character.

This condition ensures that the last line is also counted.

Step 6 – Closing the File

The try-with-resources statement automatically closes the FileInputStream.

4.5 Number of Read Calls After Buffering

The selected buffer size is 64 KB.

Buffer Size = 64 × 1024

= 65,536 bytes

For a 50 MB file:

Number of calls = File Size / Buffer Size

= 50,000,000 / 65,536

≈ 763 calls

Approximately 763 data-read calls and one final end-of-file call are required.

Comparison

Method	Approximate Read Calls
One byte at a time	50,000,000
64 KB buffer	763

The buffered approach drastically reduces the number of read operations.

The exact number of reads depends on the file size and buffer size.

4.6 Advantages of Buffering

1. Reduces repeated operating-system interactions.
2. Reads multiple bytes in a single operation.
3. Improves performance for large files.
4. Reduces repeated method-call overhead.
5. Uses a fixed amount of memory.
6. Allows large files to be processed without loading the entire file.

5. Task A2.3 – When Buffering Does Not Help (1 Mark)

Buffering does not always improve performance.

One example is a program that performs random-access operations and reads only a small amount of data from widely separated locations.

In this situation, a buffer may fetch additional bytes that the program never uses.

The extra buffering can create unnecessary memory usage and data-transfer overhead.

Another example is reading a very small file that fits into one read operation. In that case, adding a separate buffer may provide little measurable improvement.

Conclusion: Buffering is most useful for repeated sequential reading of large files. It provides less benefit when the data is small or when the application performs sparse random-access reads.

6. Alternative Concept and Complexity Analysis

6.1 Alternative – Using Files.readAllLines()

Instead of reading the file using a buffered stream, Java provides:

Files.readAllLines()

This method reads the entire text file and stores its contents in a List of Strings.

Example:

```
List<String> lines =
    Files.readAllLines(
        Path.of("attendance.log"),
        StandardCharsets.UTF_8
    );

System.out.println("Total lines: " + lines.size());
```

Working

1. Open the attendance log using Files.readAllLines().
2. Read the complete file into memory.
3. Store each line as a String in a List.
4. Use size() to calculate the total number of lines.
5. Display the result.

6.2 Complete Java Program – ReadAllLinesCounter.java

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class ReadAllLinesCounter {

    public static void main(String[] args) {

        Path file = Path.of("attendance.log");

        try {

            // Read the entire file into a List
```

```
            List<String> lines =
                Files.readAllLines(
                    file,
                    StandardCharsets.UTF_8
                );

            // Count the lines
            System.out.println(
                "Total number of lines: " + lines.size()
            );

        } catch (IOException e) {

            System.out.println(
                "File reading error: " + e.getMessage()
            );
        }
    }
}
```

Explanation:

- Path.of() identifies the file location.
- Files.readAllLines() reads the complete file.
- Each line is stored as a String.
- lines.size() returns the total number of lines.
- An IOException is handled if the file cannot be read.

6.3 Time Complexity Analysis

Let:

- N = Total number of bytes in the file.
- B = Buffer size in bytes.

- L = Total number of lines.

Approach 1 – Buffered Stream

The program reads the file once and examines every byte.

Time complexity:

O(N)

Every byte is processed exactly once.

Approach 2 – Files.readAllLines()

The program reads the complete file and creates a String for each line.

Time complexity:

O(N)

The time is proportional to the amount of file data that must be decoded and processed.

Both approaches have linear time complexity, but their memory requirements are different.

6.4 Space Complexity Analysis

Approach 1 – Buffered Stream

The program stores only:

- A 64 KB byte array.
- A few counters and variables.

Space complexity:

O(B)

For a fixed 64 KB buffer, the additional memory requirement remains approximately constant regardless of the file size.

Approach 2 – Files.readAllLines()

The entire file is stored in memory as Strings inside a List.

Space complexity:

O(N)

The actual memory usage can be greater than the original file size because of String objects, character storage, list references, and object overhead.

For a 50 MB file, the program may require substantially more than 50 MB of additional memory.

6.5 Comparison Table

Feature	Buffered Stream	Files.readAllLines()
Reading method	Reads blocks of data	Reads entire file
Time complexity	O(N)	O(N)
Space complexity	O(B)	O(N)
Memory usage	Low	High
Large file support	Suitable	May require substantial memory
Line access	Processes incrementally	All lines available in a List
Performance	Efficient for sequential reading	Convenient for small files
Best suited for	Large attendance logs	Small and moderate text files

6.6 Which Approach Is Appropriate?

Buffered Stream:

Suitable for:

- Large attendance logs.
- Millions of records.
- Line counting.
- Sequential file processing.
- Applications with limited memory.

It processes the file in chunks without storing all its contents.

Files.readAllLines():

Suitable for:

- Small text files.
- Configuration files.
- Files that require repeated access to different lines.
- Applications where all lines must be available simultaneously.

It provides simple code but requires more memory.

Engineering Decision

For the 50 MB attendance log, the buffered stream approach is recommended.

It reads data in fixed-size blocks, reduces repeated read operations, and uses only a small amount of additional memory.

Files.readAllLines() is more convenient for smaller files but is not recommended when the application only needs to count lines in a large file.

7. Final Result

The performance problem caused by reading a 50 MB attendance log one byte at a time was analysed.

The unbuffered approach requires approximately 50 million read calls, creating unnecessary method-call and operating-system overhead.

A redesigned implementation using a 64 KB byte array buffer was developed. It reads multiple bytes in each operation and counts newline characters efficiently.

The limitations of buffering were explained, and the buffered approach was compared with Files.readAllLines() using time and space complexity.

Thus, buffering provides an efficient solution for processing large sequential files while maintaining low memory usage.